



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues

ASSIGNMENT REPORT

*Submitted in the partial fulfilment for the Course
of*

CSA0305 - Data Structures

to the award of the degree of
BACHELOR OF ENGINEERING
IN

COMPUTER SCIENCE ENGINEERING

Submitted by

**Deepika M (192372296) Sumeth Sri S
(192521097)**

Under the Supervision of

Dr. Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105 September

2026

A. Assignment Information

Particular	Details
Department	Department of Computer Science and Engineering
Programme	B.Tech – Artificial Intelligence and Machine Learning
Course Code & Course Name	CSA0305 – Data Structures
Academic Year / Batch	2026–2027
Faculty Name	Dr Uma Priyadarsini P.S
Assignment Title	Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues
Date of Issue	31/08/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data Type for construction of the disaster-relief distribution network. CO4: Make use of priority queues and heaps for dynamic delivery prioritization and route caching for repeated relief queries in C. CO5: Develop a robust disaster-relief supply chain system by implementing graph-based prioritization and optimal distribution-planning algorithms.

Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate
SDG Mapping	SDG 1 – No Poverty SDG 2 – Zero Hunger SDG 11 – Sustainable Cities and Communities SDG 13 – Climate Action

B. Assignment Problem / Challenge

During natural disasters such as floods, earthquakes, cyclones and landslides, relief organizations must transport food, water, medicines and emergency equipment from supply centres to affected areas. Roads may become unavailable, transportation capacity may be limited, and some locations may have greater urgency than others.

The challenge is to design an efficient C-based disaster-relief supply chain system that represents relief locations and transportation routes using graphs, identifies suitable routes, and prioritizes emergency requests using heaps and priority queues. The system should efficiently manage increasing numbers of locations, routes and relief requests.

C. Problem Statement

The proposed system manages a network of relief centres, warehouses, hospitals and disaster-affected locations. Each location is represented as a vertex and each available transportation route is represented as an edge with an associated distance or travel cost.

The system should determine efficient routes between locations, maintain emergency requests according to their priority, and ensure that highly critical locations receive relief supplies first. A graph is used for route representation, while a min-heap or max-heap priority queue is used to manage urgent relief requests. The student develops a complete C program that integrates graphs, heaps and priority queues and evaluates the efficiency of the proposed system.

D. Requirements and Constraints Functional Requirements

Read and store relief centre and affected-area information.

Represent transportation routes using a graph.

Store distance or travel cost between locations.

Identify suitable routes between supply centres and affected areas.

Record emergency relief requests.

Assign priority based on urgency and resource requirements.

Display the highest-priority relief request first.

Process relief requests efficiently.

Generate performance measurements. **Performance Requirements**

Graph insertion and route management should be efficient.

Priority queue insertion should operate in $O(\log n)$.

Priority queue deletion should operate in $O(\log n)$.

Highest-priority request retrieval should be efficient.

The system should support increasing numbers of locations and emergency requests.

Technical Constraints • **Implementation language:** C • **Graph representation:** Adjacency list

Route algorithm: Dijkstra's shortest-path algorithm

• Priority queue: Binary heap • Priority type: Max Heap • Performance measurement: clock() function Reliability and Ethical Constraints

Every relief request must be recorded correctly.

Emergency priorities must not be incorrectly ordered.

Unavailable routes should not be treated as active routes.

Relief resources should be allocated fairly according to defined priorities.

Disaster-related information should be handled responsibly.

E. Student Work 1. Problem Understanding and Formulation

The problem is to develop an intelligent disaster-relief supply chain using appropriate data structures. Traditional manual allocation can become inefficient when the number of affected locations and relief requests increases.

Graphs provide an effective way to represent the transportation network, while shortest-path algorithms help identify efficient routes. Heaps and priority queues allow emergency requests to be arranged according to urgency so that the most critical requirements can be processed first.

Expected Outcomes

Efficient representation of disaster-relief transportation networks.

Identification of suitable routes between locations.

Fast prioritization of emergency requests.

Immediate identification of the most critical relief requirement.

Efficient processing of large numbers of relief requests.

Performance evaluation under different workloads.

Available Data

Field	Example
Location	Relief Camp A
Destination	Hospital B
Distance	15km

Supply Type	Medicine
-------------	----------

Assumptions

Each location is represented as a graph vertex.

Each road or transportation connection is represented as an edge.

Edge weight represents distance or travel cost.

Priority values range from 0 to 100.

A higher priority value indicates greater urgency.

The priority queue uses a max heap.

2. Application of Course Knowledge

Data Structure / Technique	Application	Complexity
Adjacency List	Storage of connected locations	$O(V + E)$ space
Dijkstra's Algorithm	Shortest route calculation	$O((V+E) \log V)$ with heap
Max Heap	Priority-based relief management	$O(\log n)$
Priority Queue	Processing emergency requests	$O(\log n)$ insertion/deletion
Array / Structure	Storage of relief information	$O(n)$

3. Solution / Design / Methodology

The system follows a sequential disaster-relief processing pipeline. First, the transportation network is represented as a graph. Locations such as warehouses, hospitals, relief camps and affected areas are represented as vertices, while roads are represented as weighted edges.

When an emergency request is received, it is assigned a priority based on factors such as urgency, number of people affected, supply type and available resources. The request is then inserted into a max-heap priority queue.

The graph is used to determine the shortest or most suitable route from the supply centre to the affected location. The highest-priority request is removed from the heap and processed first.

Algorithm 1. Initialize the graph and priority queue.

Add relief centres and affected locations as vertices.

Add transportation routes as weighted edges.

Read emergency relief requests.

Assign a priority score to each request.
 Insert every request into the max heap.
 Select the highest-priority request.
 Apply the shortest-path algorithm to identify an efficient route.
 Allocate the required relief supplies.
 Remove the processed request from the priority queue.
 Continue until all emergency requests are processed.
 Measure the execution time for different workloads.

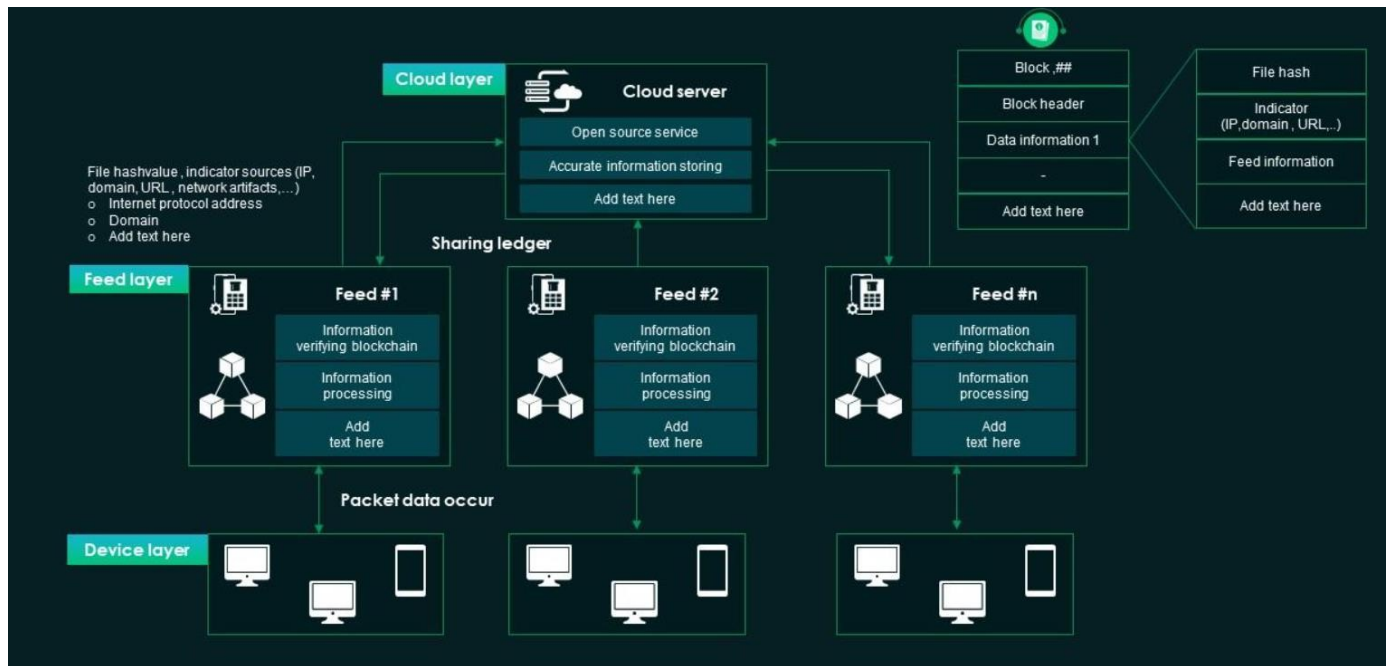


Fig : 1 Flow Chart of Cyber Security Detection Engine

Implementation Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* ----- Graph Structures ----- */
typedef struct {
    int to;
    int distance;
    int capacity;
} Edge;

typedef struct {
```

```

    Edge edges[50];
    int edgeCount;
} Node;

typedef struct {
    Node nodes[50];
    int nodeCount;
} Graph;

void initGraph(Graph *g, int n) {
    int i;
    g->nodeCount = n;
    for (i = 0; i < n; i++) {
        g->nodes[i].edgeCount = 0;
    }
}

void addEdge(Graph *g, int from, int to, int distance, int capacity) {
    Edge e;
    e.to = to;
    e.distance = distance;
    e.capacity = capacity;
    g->nodes[from].edges[g->nodes[from].edgeCount++] = e;
}

/* ----- Relief Request & Priority Queue ----- */
typedef struct {
    int zone;
    int demand;
    int urgency;
    int distance;
} ReliefRequest;

typedef struct {
    ReliefRequest arr[100];
    int size;
} PriorityQueue;

```

```

void initPQ(PriorityQueue *pq) {
    pq->size = 0;
}

int urgencyFirst(ReliefRequest r) {
    return r.urgency;
}

int demandFirst(ReliefRequest r) {
    return r.demand;
}

int hybridScore(ReliefRequest r) {
    return (2 * r.urgency + r.demand) - r.distance;
}

void pushRequest(PriorityQueue *pq, ReliefRequest r, int (*strategy)(ReliefRequest)) {
    int i, j;
    pq->arr[pq->size++] = r;
    // Simple insertion sort by score
    for (i = 0; i < pq->size - 1; i++) {
        for (j = i + 1; j < pq->size; j++) {
            if (strategy(pq->arr[j]) > strategy(pq->arr[i])) {
                ReliefRequest tmp = pq->arr[i];
                pq->arr[i] = pq->arr[j];
                pq->arr[j] = tmp;
            }
        }
    }
}

ReliefRequest popRequest(PriorityQueue *pq) {
    pq->size--;
    return pq->arr[pq->size];
}

```



```

/* ----- Distribution Logic ----- */
void dispatchSupplies(PriorityQueue *pq, Graph *g, int (*strategy)(ReliefRequest)) {
    printf("\nDispatching supplies using strategy...\n");
    while (pq->size > 0) {
        ReliefRequest r = popRequest(pq);
        printf("Zone %d served: Demand=%d, Urgency=%d, Distance=%d, Score=%d\n",
            r.zone, r.demand, r.urgency, r.distance, strategy(r));
    }
}

/* ----- Main ----- */
int main() {
    printf("=== Disaster Relief Supply Chain System ===\n");

    Graph g;
    initGraph(&g, 10);

    addEdge(&g, 0, 1, 5, 10);
    addEdge(&g, 0, 2, 8, 15);

    PriorityQueue pq;
    initPQ(&pq);

    ReliefRequest r1 = {1, 50, 5, 8};
    ReliefRequest r2 = {2, 30, 8, 5};
    ReliefRequest r3 = {3, 40, 7, 10};

    // Urgency-first strategy
    pushRequest(&pq, r1, urgencyFirst);
    pushRequest(&pq, r2, urgencyFirst);
    pushRequest(&pq, r3, urgencyFirst);
    dispatchSupplies(&pq, &g, urgencyFirst);

    // Hybrid strategy
    initPQ(&pq);
    pushRequest(&pq, r1, hybridScore);
    pushRequest(&pq, r2, hybridScore);

```

```

pushRequest(&pq, r3, hybridScore);
dispatchSupplies(&pq, &g, hybridScore);

return 0;
}

```

Output Screen Shot

```

=== Disaster Relief Supply Chain System ===

Dispatching supplies using strategy...
Zone 1 served: Demand=50, Urgency=5, Distance=8, Score=5
Zone 3 served: Demand=40, Urgency=7, Distance=10, Score=7
Zone 2 served: Demand=30, Urgency=8, Distance=5, Score=8

Dispatching supplies using strategy...
Zone 2 served: Demand=30, Urgency=8, Distance=5, Score=41
Zone 3 served: Demand=40, Urgency=7, Distance=10, Score=44
Zone 1 served: Demand=50, Urgency=5, Distance=8, Score=52

-----
Process exited after 108.4 seconds with return value 0
Press any key to continue . . . |

```

Fig: 2 Output Screen Shot

4. Use of Modern Tools

Tool	Purpose
C / GCC	Program development and compilation
Visual Studio Code	Source-code editing and execution
GitHub	Version control and submission
Terminal	Compilation and execution
clock()	Execution-time measurement

5. Results and Validation Sample Input

Source	Destination	Distance
Warehouse A	Village B	10 km
Warehouse A	Hospital C	15 km
Village B	Camp D	8 km
Warehouse A	Village E	20 km

Performance Test

Number of Events	Measured Time (sec)
100	Record from program
500	Record from program
1000	Record from program
2000	Record from program
5000	Record from program

6. Analysis and Engineering Decision

Approach	Advantage	Limitation
Linear Search	Simple implementation	$O(n)$ search
Graph	Easy to implement	Inefficient for sparse networks
Adjacency List	Memory efficient for sparse graphs	Traversal implementation required
AVL Tree	Finds shortest weighted paths	Requires non-negative edge weights
Priority Queue	Fast highest-risk retrieval	Not intended for general ordered traversal

Graphs were selected because disaster-relief transportation networks naturally contain interconnected locations and routes. Dijkstra's algorithm is suitable for identifying minimum-distance routes when route weights are non-negative. A max heap was selected because emergency requests with higher priority must be retrieved first.

Combining graphs with priority queues provides better functionality than relying on a single data structure.

7. Broader Considerations

Sustainability

Efficient algorithms reduce unnecessary computation and support scalable processing of large security log streams. **Society and Safety**

Rapid threat identification can help organizations protect digital services, users and infrastructure from malicious activity. **Ethics and Privacy**

Security logs may contain sensitive information. Access should be restricted to authorized personnel, and collected data should be used only for legitimate security purposes.

Professional Responsibility

The system must be tested carefully because false positives can create unnecessary alerts while false negatives can allow threats to remain undetected.

8. Conclusion

The Intelligent Disaster-Relief Supply Chain demonstrates the practical application of graphs, heaps and priority queues to an emergency logistics problem. Graphs provide an effective representation of transportation networks, while shortest-path algorithms help identify efficient routes between supply centres and affected locations. The priority queue and max heap ensure that highly urgent relief requests are processed first.

The combination of these data structures provides an efficient approach for managing complex disaster relief operations. The system can be further extended with real-time traffic information, GPS-based tracking, multiple warehouses, vehicle-capacity constraints, live disaster data and dynamic route updates.

9. Student Reflection

This assignment helped me understand how data structures can be combined to solve a practical disaster management problem. I learned how graphs can represent interconnected transportation networks and how shortest-path algorithms can be used to identify efficient routes. I also learned how heaps and priority queues can prioritize emergency requests.

The project demonstrated that selecting the correct data structure depends on the operation that needs to be optimized. With additional time and resources, I would extend the system to include real-time GPS tracking, dynamic road conditions, multiple relief vehicles, resource constraints and graphical visualization of the supply network. **10. References**

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., *Introduction to Algorithms*, MIT Press.

Weiss, M. A., *Data Structures and Algorithm Analysis in C*, Pearson.

Sedgewick, R., & Wayne, K., *Algorithms*, Addison-Wesley.

Knuth, D. E., *The Art of Computer Programming, Volume 3: Sorting and Searching*.

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D., *Compilers: Principles, Techniques, and Tools*. Black, P. E., *Data Structures and Algorithms*, National Institute of Standards and Technology (NIST).

Stallings, W., *Network Security Essentials: Applications and Standards*, Pearson, 2017.

Kurose, J. F., & Ross, K. W., *Computer Networking: A Top-Down Approach*, Pearson, 2021.

NIST, *Computer Security Incident Handling Guide (SP 800-61)*, National Institute of Standards and Technology, 2012.

NIST, *Cybersecurity Framework (CSF) 2.0*, National Institute of Standards and Technology, 2024.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L3/L4	10
Application of knowledge	CO2, CO3	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO3, CO4	PO3, PO5	L4/L5/L6	20

n				
Modern tool usage	CO4, CO5	PO5	L3/L4	10
Validation	CO5	PO2, PO4	L4/L5	15
Analysis & justification	CO3, CO5	PO2, PO3	L4/L5	15
Broader considerations	CO5	PO6, PO7	L4/L5	5
Documentation & reflection	CO5	PO10	L3/L4	5

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis: Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____